

Obstacle.py

```
import time
import threading
import json
import queue
import math
import numpy as np
import pytsx3

from flask import Flask, request, jsonify, send_from_directory, Response,
stream_with_context

from collections import deque
import cv2
from ultralytics import YOLO
import os
import socket
import requests


app = Flask(__name__)


current_location = None
current_location_lock = threading.Lock()


def _collect_lan_ipv4_addresses():
    """Best-effort LAN IPv4 discovery for phone access hints."""
    addresses = set()


    try:
        host_name = socket.gethostname()
```

```
for family, _, _, sockaddr in socket.getaddrinfo(host_name, None):
    if family == socket.AF_INET:
        ip = sockaddr[0]
        if ip and not ip.startswith("127.") and not ip.startswith("169.254."):
            addresses.add(ip)
    except Exception:
        pass
```

```
try:
    probe = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    probe.connect(("8.8.8.8", 80))
    ip = probe.getsockname()[0]
    probe.close()
    if ip and not ip.startswith("127.") and not ip.startswith("169.254."):
        addresses.add(ip)
except Exception:
    pass
```

```
return sorted(addresses)
```

```
def _recommended_urls(port=5000, https_enabled=True):
    scheme = "https" if https_enabled else "http"
    lan_ips = _collect_lan_ipv4_addresses()
    lan_urls = [f'{scheme}://{ip}:{port}/movements' for ip in lan_ips]
    return {
        "scheme": scheme,
        "port": port,
```

```

"lan_ips": lan_ips,
"lan_urls": lan_urls,
"localhost_url": f'{scheme}://127.0.0.1:{port}/movements',
"usb_adb_reverse_command": f'adb reverse tcp:{port} tcp:{port}',
"usb_phone_url": f'http://127.0.0.1:{port}/movements',
}

```

Turn-by-turn navigation (live GPS, route progression, voice for maneuvers) is implemented

in movements.html: it uses watchPosition + OSRM geometry; this file serves /sensor, camera,

TTS/SSE, and SOS. No server-side route geometry is required for that flow.

=====

SSE — real-time push to phone

=====

```
_sse_clients = []
```

```
_sse_clients_lock = threading.Lock()
```

```
def broadcast_sse(event_type, data):
```

```
    msg = f'event: {event_type}\n' + f'data: {json.dumps(data)}\n\n'
```

```
    with _sse_clients_lock:
```

```
        dead = []
```

```
        for client_queue in _sse_clients:
```

```
            try:
```

```
                client_queue.put_nowait(msg)
```

```
            except Exception:
```

```
                dead.append(client_queue)
```

```

for dead_queue in dead:
    _sse_clients.remove(dead_queue)

@app.route("/events")
def sse_stream():
    client_queue = queue.Queue(maxsize=30)
    with _sse_clients_lock:
        _sse_clients.append(client_queue)

    def generate():
        yield "event: connected\ndata: {}\n\n"
        while True:
            try:
                msg = client_queue.get(timeout=20)
                yield msg
            except queue.Empty:
                yield ": keepalive\n\n"

    response = Response(stream_with_context(generate()), mimetype="text/event-stream")
    response.headers["Cache-Control"] = "no-cache"
    response.headers["X-Accel-Buffering"] = "no"
    response.headers["Access-Control-Allow-Origin"] = "*"
    return response

# =====
# TEXT TO SPEECH
# =====

```

```

# =====

tts_engine = pyttsx3.init()
tts_engine.setProperty('rate', 155)
tts_engine.setProperty('volume', 1.0)
voices = tts_engine.getProperty('voices')
for v in voices:
    if 'zira' in v.name.lower() or 'david' in v.name.lower():
        tts_engine.setProperty('voice', v.id)
        break

tts_lock = threading.Lock()
_speech_queue = deque()
_speech_queue_lock = threading.Lock()
_speech_event = threading.Event()

def _enqueue(text, priority=False):
    with _speech_queue_lock:
        if text in _speech_queue:
            return
        if priority:
            _speech_queue.appendleft(text)
        else:
            _speech_queue.append(text)
        _speech_event.set()

def _tts_worker():
    while True:

```

```

    _speech_event.wait()
while True:
    with _speech_queue_lock:
        if not _speech_queue:
            _speech_event.clear()
            break
        text = _speech_queue.popleft()
        with tts_lock:
            try:
                tts_engine.say(text)
                tts_engine.runAndWait()
            except Exception:
                pass

threading.Thread(target=_tts_worker, daemon=True).start()

def speak_movement(text):
    broadcast_sse("speak", {"text": text})
    _enqueue(text, priority=False)

def speak_obstacle(text):
    broadcast_sse("speak_urgent", {"text": text})
    _enqueue(text, priority=True)

def speak_force(text):
    broadcast_sse("speak_urgent", {"text": text})
    _enqueue(text, priority=True)

```

```

def _format_distance(meters):
    if meters is None:
        return "a short distance"
    if meters >= 1000:
        return f"{round(meters / 1000, 2)} km"
    return f"{int(round(meters))} m"

```

```

def _haversine_meters(lat1, lon1, lat2, lon2):
    r = 6371000.0
    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlambda = math.radians(lon2 - lon1)

```

```

    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) *
        math.sin(dlambda / 2) ** 2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return r * c

```

```

def _bearing_to_cardinal(lat1, lon1, lat2, lon2):
    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    dlambda = math.radians(lon2 - lon1)

```

```

    y = math.sin(dlambda) * math.cos(phi2)
    x = math.cos(phi1) * math.sin(phi2) - math.sin(phi1) * math.cos(phi2) *
        math.cos(dlambda)

```

```
bearing = (math.degrees(math.atan2(y, x)) + 360) % 360
dirs = ["north", "north-east", "east", "south-east", "south", "south-west", "west",
"north-west"]
return dirs[int((bearing + 22.5) / 45) % 8]
```

```
def _format_osrm_step(step):
maneuver = step.get("maneuver", {})
m_type = maneuver.get("type", "continue")
modifier = maneuver.get("modifier", "").strip()
road = (step.get("name") or "").strip()
dist = _format_distance(step.get("distance", 0))
```

```
road_text = f" on {road}" if road else ""
```

```
if m_type == "depart":
if modifier:
return f"Start and head {modifier} {road_text} for {dist}."
return f"Start and continue {road_text} for {dist}."
```

```
if m_type == "arrive":
return "You have arrived at your destination."
```

```
if m_type == "roundabout":
return f"At the roundabout, follow the exit {road_text} and continue for {dist}."
```

```
if m_type == "new name":
return f"Continue {road_text} for {dist}."
```

```
if modifier in {"left", "slight left", "sharp left"}:
    return f"Turn {modifier} {road_text}, then continue for {dist}."
if modifier in {"right", "slight right", "sharp right"}:
    return f"Turn {modifier} {road_text}, then continue for {dist}."
if modifier == "uturn":
    return f"Take a U-turn{road_text}, then continue for {dist}."
```

```
if m_type == "continue":
    return f"Continue {road_text} for {dist}."
```

```
return f"Go ahead {road_text} for {dist}."
```

```
def _geocode_destination(destination_text):
    if not destination_text:
        raise ValueError("Destination is required")
```

```
url = "https://nominatim.openstreetmap.org/search"
headers = {"User-Agent": "blind-navigation-vedha/1.0"}
```

```
queries = [
    f"{destination_text}, Tamil Nadu, India",
    destination_text,
]
```

```
for query in queries:
    response = requests.get(
        url,
```

```
params={"q": query, "format": "jsonv2", "limit": 1},
headers=headers,
timeout=12,
)
response.raise_for_status()
items = response.json()
if items:
    item = items[0]
    return float(item["lat"]), float(item["lon"]), item.get("display_name",
destination_text)
```

```
raise ValueError(f'Destination not found: {destination_text}')
```

```
def _fetch_osrm_route(start_lat, start_lon, end_lat, end_lon):
    url = f'https://router.project-
osrm.org/route/v1/foot/{start_lon},{start_lat};{end_lon},{end_lat}'
    response = requests.get(
        url,
        params={
            "overview": "full",
            "geometries": "geojson",
            "steps": "true",
            "alternatives": "false",
        },
        timeout=15,
    )
    response.raise_for_status()
    payload = response.json()
```

```
if payload.get("code") != "Ok" or not payload.get("routes"):
    raise RuntimeError("Route service did not return a valid route")
```

```
route = payload["routes"][0]
distance_m = float(route.get("distance", 0.0))
duration_s = float(route.get("duration", 0.0))
```

```
geometry = route.get("geometry", {}).get("coordinates", [])
route_coords = [[lat, lon] for lon, lat in geometry]
```

```
steps = []
for leg in route.get("legs", []):
    for step in leg.get("steps", []):
        steps.append(_format_osrm_step(step))
```

```
if not steps:
    steps = ["Proceed towards destination.", "You have arrived at your destination."]
elif "arrived" not in steps[-1].lower():
    steps.append("You have arrived at your destination.")
```

```
return {
    "distance_m": distance_m,
    "duration_s": duration_s,
    "steps": steps,
    "route_coords": route_coords,
}
```

```

def _build_fallback_route(start_lat, start_lon, end_lat, end_lon,
destination_name):

    distance_m = _haversine_meters(start_lat, start_lon, end_lat, end_lon)
    duration_s = distance_m / 1.2 if distance_m > 0 else 0.0
    direction = _bearing_to_cardinal(start_lat, start_lon, end_lat, end_lon)

    steps = [
        f"From your current location, head {direction}.",
        f"Continue for about {_format_distance(distance_m)}.",
        f"Reach destination: {destination_name}.",
    ]

    return {
        "distance_m": distance_m,
        "duration_s": duration_s,
        "steps": steps,
        "route_coords": [[start_lat, start_lon], [end_lat, end_lon]],
    }

# =====
# OBSTACLE TRACKER — max 2 announcements, then periodic
# =====

RE_ANNOUNCE_SEC = 8.0
MAX_FIRST_BURST = 2

class ObstacleTracker:
    def __init__(self):

```

```

self.lock          = threading.Lock()
self.current_text  = ""
self.announce_count = 0
self.last_said_time = 0.0

def should_speak(self, text):
    with self.lock:
        now = time.time()
        if text != self.current_text:
            self.current_text = text
            self.announce_count = 1
            self.last_said_time = now
        return True
    if self.announce_count < MAX_FIRST_BURST:
        if (now - self.last_said_time) >= 1.5:
            self.announce_count += 1
            self.last_said_time = now
        return True
    return False

    if (now - self.last_said_time) >= RE_ANNOUNCE_SEC:
        self.last_said_time = now
        return True
    return False

def clear(self):
    with self.lock:
        self.current_text = ""

```

```
self.announce_count = 0
```

```
obs_tracker = ObstacleTracker()
```

```
# =====
```

```
# YOLO CONFIG
```

```
# =====
```

```
OBJECT_REF = {
```

```
0 :(200,"person"), 1:(80,"bicycle"), 2:(130,"car"),
```

```
3 :(110,"motorbike"),5:(120,"bus"), 7:(120,"truck"),
```

```
9 :(60,"traffic light"), 11:(90,"stop sign"), 13:(60,"bench"),
```

```
24:(50,"backpack"), 56:(70,"chair"), 57:(90,"couch"),
```

```
58:(90,"potted plant"), 60:(60,"dining table"),
```

```
62:(80,"TV"), 63:(50,"laptop"), 67:(50,"phone"),
```

```
72:(60,"refrigerator"),
```

```
}
```

```
DEFAULT_REF_HEIGHT = 80
```

```
DANGER_CLASSES = {0,1,2,3,5,7}
```

```
ZONE_CRITICAL = 1.0
```

```
ZONE_CLOSE = 2.5
```

```
ZONE_WARN = 4.0
```

```
def estimate_distance(box_h_px, class_id, frame_h):
```

```
ref_h, _ = OBJECT_REF.get(class_id, (DEFAULT_REF_HEIGHT, "object"))
```

```
scale = frame_h / 480.0
```

```
ref_h *= scale
```

```
return 99.0 if box_h_px < 1 else round(ref_h / box_h_px, 1)
```

```
def horizontal_zone(obj_cx, frame_w):
```

```
    r = obj_cx / frame_w
```

```
    if r < 0.20: return "far_left"
```

```
    elif r < 0.40: return "left"
```

```
    elif r < 0.60: return "center"
```

```
    elif r < 0.80: return "right"
```

```
    else:      return "far_right"
```

```
def build_instruction(label, position, distance):
```

```
    dist_str = f"{distance} metres" if distance < 10 else "some distance away"
```

```
    if distance <= ZONE_CRITICAL:
```

```
        urgency = 3
```

```
        if position == "center":
```

```
            return f"Stop! {label} directly in front, less than one metre.", urgency
```

```
        elif "left" in position:
```

```
            return f"Danger! {label} very close on your left. Step right immediately.",  
                    urgency
```

```
        else:
```

```
            return f"Danger! {label} very close on your right. Step left immediately.",  
                    urgency
```

```
    elif distance <= ZONE_CLOSE:
```

```
        urgency = 2
```

```
        if position == "center":
```

```
            return f"Caution. {label} ahead, about {dist_str}. Slow down.", urgency
```

```
        elif "left" in position:
```

```
            return f"{label} on your left at {dist_str}. Move right.", urgency
```

```

else:
    return f"{label} on your right at {dist_str}. Move left.", urgency
else:
    urgency = 1
    if position == "center":
        return f"{label} ahead at {dist_str}. Be careful.", urgency
    elif "left" in position:
        return f"{label} on your left, {dist_str}.", urgency
    else:
        return f"{label} on your right, {dist_str}.", urgency

```

```

def pick_most_dangerous(detections):
    if not detections: return None
    detections.sort(key=lambda d: (0 if d[0] in DANGER_CLASSES else 1, d[1]))
    return detections[0]

```

```

# =====
# MOVEMENT THRESHOLDS
# =====

STILL_MAX          = 0.04
STANDING_MAX       = 0.40
WALKING_PEAKS_MIN  = 3
PEAK_THRESHOLD_OFFSET = 0.20
MIN_PEAK_GAP       = 6
IDLE_TIMEOUT_SEC   = 8.0
MAJORITY_THRESHOLD = 0.6
CONFIRM_WINDOW     = 5

```

```

# =====
# GLOBAL STATE
# =====

movement_state      = "IDLE"
state_lock           = threading.Lock()
last_confirmed_state = "IDLE"
last_state_change_time = time.time()
prediction_buffer     = deque(maxlen=CONFIRM_WINDOW)


# — Obstacle state exposed to dashboard —————
obstacle_state = {
    "camera_active" : False,
    "last_obstacle" : "",
    "last_urgency"   : 0,
    "last_update"    : 0.0,
}
obstacle_lock = threading.Lock()


sos_state = {
    "last_sent_at": 0.0,
    "last_reason": "",
    "last_location": None,
    "last_contacts": 0,
}
sos_lock = threading.Lock()

```

```

STATE_VOICE = {
"WALKING" : "Walking. Obstacle detection active.",
"STANDING" : "You have stopped.",
"SITTING" : "You are seated.",
"IDLE"    : "No movement detected.",
}

# =====
# CAMERA THREAD
# =====

model      = YOLO("yolov8n.pt")
model_lock = threading.Lock()
camera_active = False
camera_thread = None
camera_lock = threading.Lock()

clear_path_counter = 0
last_clear_announce = 0.0
CLEAR_PATH_FRAMES = 8
CLEAR_ANNOUNCE_GAP = 12.0

# For phone camera stream path-clear announcements
phone_clear_counter = 0
phone_last_clear_announce = 0.0

def run_yolo_inference(frame, conf=0.30):
    """Run YOLO safely and recover once from known fuse/bn compatibility
    errors."""

```

global model

with model_lock:

try:

return model(frame, verbose=False, conf=conf)

except AttributeError as err:

msg = str(err)

if "has no attribute 'bn'" not in msg:

raise

print("⚠ YOLO fuse error detected. Reloading model and retrying once...")

model = YOLO("yolov8n.pt")

return model(frame, verbose=False, conf=conf)

def obstacle_detection_loop():

global camera_active, clear_path_counter, last_clear_announce

cap = cv2.VideoCapture(0)

if not cap.isOpened():

print("✖ Camera not opened")

camera_active = False

with obstacle_lock:

obstacle_state["camera_active"] = False

return

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

```
cap.set(cv2.CAP_PROP_FPS, 15)
```

```
print("📷 Obstacle detection started")
```

```
obs_tracker.clear()
```

```
clear_path_counter = 0
```

```
with obstacle_lock:
```

```
    obstacle_state["camera_active"] = True
```

```
    obstacle_state["last_obstacle"] = ""
```

```
    obstacle_state["last_urgency"] = 0
```

```
while camera_active:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        time.sleep(0.5)
```

```
        continue
```

```
    results = run_yolo_inference(frame, conf=0.45)
```

```
    if not camera_active:
```

```
        break
```

```
    h, w, _ = frame.shape
```

```
    detections = []
```

```
    for r in results:
```

```
        for box in r.bboxes:
```

```

conf    = float(box.conf[0])
class_id = int(box.cls[0])
if conf < 0.45: continue
x1, y1, x2, y2 = map(int, box.xyxy[0])
box_h    = y2 - y1
obj_cx   = (x1 + x2) // 2
distance = estimate_distance(box_h, class_id, h)
position = horizontal_zone(obj_cx, w)
_, lbl   = OBJECT_REF.get(class_id, (DEFAULT_REF_HEIGHT, "obstacle"))
if distance <= ZONE_WARN:
detections.append((class_id, distance, position, lbl))

if detections:
clear_path_counter = 0
best = pick_most_dangerous(detections)
if best:
cls_id, dist, pos, lbl = best
instruction, urgency   = build_instruction(lbl, pos, dist)

# Update dashboard state always (even if suppressed in TTS)
with obstacle_lock:
obstacle_state["last_obstacle"] = instruction
obstacle_state["last_urgency"]   = urgency
obstacle_state["last_update"]   = time.time()

if obs_tracker.should_speak(instruction):
speak_obstacle(instruction)

```

```

print(f"🚧 [{urgency}★] {instruction}")
else:
print(f"🔇 [{urgency}★] suppressed: {instruction}")
else:
obs_tracker.clear()
clear_path_counter += 1

# Update dashboard — clear obstacle
with obstacle_lock:
obstacle_state["last_obstacle"] = ""
obstacle_state["last_urgency"] = 0

now = time.time()
if (clear_path_counter >= CLEAR_PATH_FRAMES and
(now - last_clear_announce) > CLEAR_ANNOUNCE_GAP):
speak_obstacle("Path is clear. Continue walking.")
last_clear_announce = now
clear_path_counter = 0
print("✅ Path clear")
with obstacle_lock:
obstacle_state["last_obstacle"] = "Path is clear"
obstacle_state["last_urgency"] = 0

time.sleep(0.35)

cap.release()
obs_tracker.clear()

```

```

with obstacle_lock:
obstacle_state["camera_active"] = False
obstacle_state["last_obstacle"] = ""
obstacle_state["last_urgency"] = 0
print("📷 Obstacle detection stopped")

def manage_camera(final_state):
    """Update camera state for frontend to react to"""
    global camera_active
    with camera_lock:
        if final_state == "WALKING" and not camera_active:
            camera_active = True
            with obstacle_lock:
                obstacle_state["camera_active"] = True
            print("📷 Camera should be active (frontend)")
        elif final_state != "WALKING" and camera_active:
            camera_active = False
            with obstacle_lock:
                obstacle_state["camera_active"] = False
            print("📷 Camera should be inactive (frontend)")

```

```

# =====
# CLASSIFIER
# =====

def classify(window: np.ndarray) -> tuple:
    magnitude = np.linalg.norm(window, axis=1)
    mag_std = float(np.std(magnitude))

```

```

dynamic = magnitude - np.mean(magnitude)
peaks = 0
last_peak = -MIN_PEAK_GAP
for j in range(1, len(dynamic) - 1):
    if (dynamic[j] > dynamic[j-1] and dynamic[j] > dynamic[j+1] and
        dynamic[j] > PEAK_THRESHOLD_OFFSET and (j - last_peak) >=
        MIN_PEAK_GAP):
        peaks += 1
        last_peak = j
    if mag_std < STILL_MAX:
        return "SITTING", mag_std, peaks
    if mag_std < STANDING_MAX:
        return ("WALKING" if peaks >= WALKING_PEAKS_MIN else
            "STANDING"), mag_std, peaks
    return ("WALKING" if peaks >= WALKING_PEAKS_MIN else
        "STANDING"), mag_std, peaks

# =====
# STATE MACHINE
# =====

def resolve_state(candidate: str) -> tuple:
    global last_confirmed_state, last_state_change_time
    now = time.time()
    prediction_buffer.append(candidate)
    counts = {s: prediction_buffer.count(s) for s in set(prediction_buffer)}
    top_state = max(counts, key=counts.get)
    top_ratio = counts[top_state] / len(prediction_buffer)

```

```
if top_ratio < MAJORITY_THRESHOLD:
```

```
    return last_confirmed_state, top_ratio
```

```
if top_state == "STANDING":
```

```
    elapsed = now - last_state_change_time
```

```
    if last_confirmed_state in ("STANDING", "IDLE") and elapsed >=
        IDLE_TIMEOUT_SEC:
```

```
        top_state = "IDLE"
```

```
if top_state != last_confirmed_state:
```

```
    last_confirmed_state = top_state
```

```
    last_state_change_time = now
```

```
    print(f"🔄 State → {top_state} (ratio={top_ratio:.2f})")
```

```
    return top_state, top_ratio
```

```
# =====
```

```
# FLASK ENDPOINTS
```

```
# =====
```

```
@app.route("/")
```

```
def home():
```

```
    """Landing page with menu to choose features"""
```

```
    return send_from_directory(".", "home.html")
```

```
@app.route("/movements")
```

```
def movements_page():
```

```
    """Advanced combined page with all features"""
```

```
    return send_from_directory(".", "movements.html")
```

```
@app.route("/manifest.json")
def manifest():
    return send_from_directory(".", "manifest.json")
```

```
@app.route("/sw.js")
def service_worker():
    return send_from_directory(".", "sw.js")
```

```
# Serve frontend folder files
@app.route("/frontend/<path:filename>")
def serve_frontend(filename):
    frontend_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..",
                                "frontend")
    return send_from_directory(frontend_dir, filename)
```

```
@app.route("/location", methods=["POST"])
def set_current_location():
    global current_location

    data = request.get_json(silent=True) or {}
    if "lat" not in data or "lon" not in data:
        return jsonify({"status": "error", "message": "lat and lon are required"}), 400

    try:
        lat = float(data["lat"])
        lon = float(data["lon"])
    except (TypeError, ValueError):
```

```
return jsonify({"status": "error", "message": "lat/lon must be numeric"}), 400
```

```
with current_location_lock:
```

```
current_location = (lat, lon)
```

```
broadcast_sse("location_update", {"lat": lat, "lon": lon})
```

```
return jsonify({"status": "success", "location": {"lat": lat, "lon": lon}})
```

```
@app.route("/current_location", methods=["GET"])
```

```
def get_current_location():
```

```
with current_location_lock:
```

```
loc = current_location
```

```
if loc is None:
```

```
return jsonify({
```

```
"status": "no_location",
```

```
"message": "Current location not set yet. Send location to /location first.",
```

```
})
```

```
return jsonify({"status": "success", "lat": loc[0], "lon": loc[1]})
```

```
@app.route("/navigate", methods=["POST"])
```

```
@app.route("/api/navigate", methods=["POST"])
```

```
def navigate_route():
```

```
data = request.get_json(silent=True) or {}
```

```
with current_location_lock:
```

```
stored_location = current_location
```

```
try:
```

```
start_lat = float(data.get("lat", data.get("latitude", stored_location[0] if  
stored_location else None)))
```

```
start_lon = float(data.get("lon", data.get("longitude", stored_location[1] if  
stored_location else None)))
```

```
except (TypeError, ValueError):
```

```
return jsonify({
```

```
"status": "error",
```

```
"message": "Valid current location is required. Send lat/lon or call /location  
first.",
```

```
}), 400
```

```
destination_name = str(data.get("destination", "")).strip()
```

```
try:
```

```
if "dest_lat" in data and "dest_lon" in data:
```

```
dest_lat = float(data["dest_lat"])
```

```
dest_lon = float(data["dest_lon"])
```

```
resolved_name = destination_name or "Destination"
```

```
else:
```

```
dest_lat, dest_lon, resolved_name = _geocode_destination(destination_name)
```

```
if not destination_name:
```

```
destination_name = resolved_name
```

```
try:
```

```
route_data = _fetch_osrm_route(start_lat, start_lon, dest_lat, dest_lon)
```

```

source = "osrm"

except Exception:

route_data = _build_fallback_route(start_lat, start_lon, dest_lat, dest_lon,
resolved_name)

source = "fallback"


distance_km = round(route_data["distance_m"] / 1000, 2)
duration_min = round(route_data["duration_s"] / 60, 1)


if route_data["steps"]:
speak_movement(f"Route ready. Distance is {distance_km} kilometer.")


return jsonify({
    "status": "success",
    "source": source,
    "current_location": {"lat": start_lat, "lon": start_lon},
    "destination": {
        "name": resolved_name,
        "lat": dest_lat,
        "lon": dest_lon,
    },
    "distance_km": distance_km,
    "duration_min": duration_min,
    "steps": route_data["steps"],
    "route_coords": route_data["route_coords"],
})

except ValueError as err:

return jsonify({"status": "error", "message": str(err)}), 400

```

```
except requests.RequestException as err:
    return jsonify({"status": "error", "message": f"Network error: {err}"}), 503
except Exception as err:
    return jsonify({"status": "error", "message": f"Navigation failed: {err}"}), 500
```

```
@app.route("/sensor", methods=["POST"])
```

```
def sensor_api():
```

```
    global movement_state
```

```
    data = request.json.get("data", [])
```

```
    if len(data) < 128:
```

```
        return jsonify({"movement": movement_state, "status": "buffering",
                        "samples_received": len(data)})
```

```
    window = np.array(data[-128:], dtype=np.float32)
```

```
    if window.ndim != 2 or window.shape[1] != 3:
```

```
        return jsonify({"error": "Expected shape (128, 3)"}), 400
```

```
    candidate, mag_std, peaks = classify(window)
```

```
    final_state, ratio = resolve_state(candidate)
```

```
    with state_lock:
```

```
        prev_state = movement_state
```

```
        movement_state = final_state
```

```
    if final_state != prev_state:
```

```
        speak_movement(STATE_VOICE[final_state])
```

```
manage_camera(final_state)
```

```
print(f'👁 {final_state:10s} | candidate={candidate:10s} | "  
f"mag_std={mag_std:.3f} | peaks={peaks} | ratio={ratio:.2f}")
```

```
with obstacle_lock:
```

```
obs = dict(obstacle_state)
```

```
return jsonify({  
    "movement"    : final_state,  
    "candidate"    : candidate,  
    "mag_std"      : round(mag_std, 3),  
    "peaks"        : peaks,  
    "ratio"        : round(ratio, 2),  
    "timestamp"    : round(time.time(), 2),  
    "camera_active" : obs["camera_active"],  
    "last_obstacle" : obs["last_obstacle"],  
    "last_urgency"  : obs["last_urgency"],  
})
```

```
@app.route("/network-info")
```

```
def network_info():
```

```
    script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
    cert_file = os.path.join(script_dir, "cert.pem")
```

```
    key_file = os.path.join(script_dir, "key.pem")
```

```
    https_enabled = os.path.exists(cert_file) and os.path.exists(key_file)
```

```
return jsonify(_recommended_urls(port=5000, https_enabled=https_enabled))
```

```
@app.route("/status")
```

```
def status():
```

```
    with obstacle_lock:
```

```
        obs = dict(obstacle_state)
```

```
    return jsonify({
```

```
        "server"      : "running",
```

```
        "state"       : movement_state,
```

```
        "camera_active" : obs["camera_active"],
```

```
        "last_obstacle" : obs["last_obstacle"],
```

```
        "last_urgency"  : obs["last_urgency"],
```

```
        "last_update"   : obs["last_update"],
```

```
    })
```

```
@app.route("/thresholds")
```

```
def thresholds():
```

```
    return jsonify({
```

```
        "STILL_MAX": STILL_MAX, "STANDING_MAX": STANDING_MAX,
```

```
        "WALKING_PEAKS_MIN": WALKING_PEAKS_MIN,
```

```
        "ZONE_CRITICAL_m": ZONE_CRITICAL, "ZONE_CLOSE_m":  
        ZONE_CLOSE,
```

```
        "ZONE_WARN_m": ZONE_WARN
```

```
    })
```

```
@app.route("/camera_frame", methods=["POST"])
```

```
def camera_frame():
```

```
    """Receive camera frame from frontend and process for obstacles"""
```

```
global phone_clear_counter, phone_last_clear_announce
try:
# Get the image data from request
frame_data = request.get_data()
if not frame_data:
return jsonify({"error": "No frame data"}), 400

# Convert bytes to numpy array
nparr = np.frombuffer(frame_data, np.uint8)
frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

if frame is None:
return jsonify({"error": "Invalid image"}), 400

# Process frame with YOLO
results = run_yolo_inference(frame, conf=0.30) # Lower confidence threshold
to 0.30 for better detection
h, w, _ = frame.shape
detections = []

total_detections = 0
for r in results:
for box in r.bboxes:
total_detections += 1
conf = float(box.conf[0])
class_id = int(box.cls[0])

# Log all detections for debugging
```

```

class_name = OBJECT_REF.get(class_id, (None, "unknown"))[1]
print(f" Detected: {class_name} (ID:{class_id}) - Confidence: {conf:.2f}")

if conf < 0.30: # Skip very low confidence detections
    continue

x1, y1, x2, y2 = map(int, box.xyxy[0])
box_h = y2 - y1
obj_cx = (x1 + x2) // 2
distance = estimate_distance(box_h, class_id, h)
position = horizontal_zone(obj_cx, w)
_, lbl = OBJECT_REF.get(class_id, (DEFAULT_REF_HEIGHT, "obstacle"))

# Detect any object within ZONE_WARN (4.0 meters), not just
DANGER_CLASSES
if distance <= ZONE_WARN:
    detections.append((class_id, distance, position, lbl, conf))
    print(f" ⚠ {lbl} at {distance}m, {position} (conf: {conf:.2f})")

if detections:
    phone_clear_counter = 0
    # Sort by distance to get the closest obstacle
    detections.sort(key=lambda x: x[1])
    cls_id, dist, pos, lbl, conf = detections[0]

instruction, urgency = build_instruction(lbl, pos, dist)

# Update obstacle state

```

```

with obstacle_lock:
    obstacle_state["last_obstacle"] = instruction
    obstacle_state["last_urgency"] = urgency
    obstacle_state["last_update"] = time.time()

# Speak if needed
if obs_tracker.should_speak(instruction):
    speak_obstacle(instruction)
    print(f"🚧 [{urgency} ★] {instruction}")

return jsonify({
    "status": "ok",
    "obstacle": {
        "text": instruction,
        "urgency": urgency
    }
})
else:
    # No obstacles detected within warning zone
    obs_tracker.clear()
    phone_clear_counter += 1
    with obstacle_lock:
        obstacle_state["last_obstacle"] = ""
        obstacle_state["last_urgency"] = 0

    now = time.time()
    if (phone_clear_counter >= 6 and

```

```

(now - phone_last_clear_announce) >= CLEAR_ANNOUNCE_GAP):
speak_obstacle("Path is clear. Continue walking.")
phone_last_clear_announce = now
phone_clear_counter = 0
with obstacle_lock:
obstacle_state["last_obstacle"] = "Path is clear. Continue walking."
obstacle_state["last_urgency"] = 0

# Debug info
if total_detections > 0:
print(f" ⚠ Detected {total_detections} object(s) but none within
{ZONE_WARN}m")
else:
print(f" ✓ No obstacles detected")

return jsonify({
"status": "ok",
"obstacle": {
"text": "Path clear. No obstacles.",
"urgency": 0
}
})

except Exception as e:
print(f" ✖ Error processing camera frame: {e}")
import traceback
traceback.print_exc()

```

```
return jsonify({"error": str(e)}), 500
```

```
@app.route("/camera/start", methods=["POST"])
```

```
def camera_start():
```

```
    """Endpoint to acknowledge camera start (frontend handles camera)"""
```

```
    return jsonify({"status": "ok", "message": "Camera ready"})
```

```
def _contacts_candidates():
```

```
    script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
    return [
```

```
        os.path.join(script_dir, "contacts.json"),
```

```
        os.path.join(os.path.dirname(script_dir), "contacts.json"),
```

```
    ]
```

```
def load_trusted_contacts():
```

```
    for path in _contacts_candidates():
```

```
        if not os.path.exists(path):
```

```
            continue
```

```
        try:
```

```
            with open(path, "r", encoding="utf-8") as f:
```

```
                raw = json.load(f)
```

```
                contacts = []
```

```
                if isinstance(raw, dict):
```

```
                    for name, phone in raw.items():
```

```
                        contacts.append({"name": str(name), "phone": str(phone)})
```

```
                elif isinstance(raw, list):
```

```
                    for item in raw:
```

```

if isinstance(item, dict):
    name = str(item.get("name", "Unknown"))
    phone = str(item.get("phone", ""))
    if phone:
        contacts.append({"name": name, "phone": phone})
    return contacts
except Exception as err:
    print(f'⚠ Failed to load contacts from {path}: {err}')
    return []

```

```

@app.route("/sos/contacts", methods=["GET"])
def sos_contacts():
    contacts = load_trusted_contacts()
    return jsonify({"count": len(contacts), "contacts": contacts})

```

```

# Frontend compatibility aliases
@app.route("/api/contacts", methods=["GET"])
def api_contacts():
    """Alias for /sos/contacts - used by frontend"""
    contacts = load_trusted_contacts()
    return jsonify(contacts)

```

```

@app.route("/sos", methods=["POST"])
def sos_alert():
    data = request.get_json(silent=True) or {}
    reason = str(data.get("reason", "Emergency assistance needed")).strip() or "Emergency assistance needed"
    location = data.get("location")

```

```

contacts = load_trusted_contacts()
with sos_lock:
    sos_state["last_sent_at"] = time.time()
    sos_state["last_reason"] = reason
    sos_state["last_location"] = location
    sos_state["last_contacts"] = len(contacts)

speak_force("Emergency alert sent")
print("📢 SOS ALERT RECEIVED")
print(f" Reason: {reason}")
if isinstance(location, dict):
    print(f" Location: {location.get('lat')}, {location.get('lon')}")

primary_contact = contacts[0] if contacts else None

if contacts:
    print(" Notifying trusted contacts:")
    for c in contacts:
        print(f" - {c['name']} ({c['phone']})")
    else:
        print(" No trusted contacts found in contacts.json")

return jsonify({
    "status": "success",
    "message": "SOS alert triggered",
    "contacts_notified": len(contacts),

```

```

"primary_contact": primary_contact,
"reason": reason,
"location": location,
"timestamp": round(time.time(), 2),
})

```

```

@app.route("/api/sos", methods=["POST"])

```

```

def api_sos():

```

```

    """Alias for /sos - used by frontend"""

```

```

    return sos_alert()

```

```

# =====

```

```

# MAIN

```

```

# =====

```

```

if __name__ == "__main__":

```

```

    speak_force("Navigation system ready. Please start moving.")

```

```

# Check for HTTPS certificates

```

```

script_dir = os.path.dirname(os.path.abspath(__file__))

```

```

cert_file = os.path.join(script_dir, "cert.pem")

```

```

key_file = os.path.join(script_dir, "key.pem")

```

```

# Generate certificates if they don't exist

```

```

if not (os.path.exists(cert_file) and os.path.exists(key_file)):

```

```

    print("🔒 Generating SSL certificates for HTTPS...\n")

```

```

    try:

```

```

        from generate_certificates import generate_certificates

```

```

generate_certificates()
except Exception as e:
    print(f'⚠️ Could not generate certificates: {e}\n')

protocol = "https" if (os.path.exists(cert_file) and os.path.exists(key_file)) else
"http"

urls = _recommended_urls(port=5000, https_enabled=(protocol == "https"))

print("🌀 Blind Navigation Backend — port 5000")
print("—" * 60)

print(f'📍 SITTING → mag_std < {STILL_MAX}')
print(f'📍 STANDING → mag_std {STILL_MAX}–{STANDING_MAX},
peaks < {WALKING_PEAKS_MIN}')
print(f'📍 WALKING → mag_std > {STANDING_MAX} AND peaks >=
{WALKING_PEAKS_MIN}')
print("—" * 60)

print(f'🚫 CRITICAL : ≤ {ZONE_CRITICAL} m → STOP immediately')
print(f'🚫 CLOSE : ≤ {ZONE_CLOSE} m → slow down / turn')
print(f'🚫 WARN : ≤ {ZONE_WARN} m → heads-up')
print("—" * 60)

print(f'\n📱 Open on your PHONE (same Wi-Fi):')
if urls["lan_urls"]:
    for url in urls["lan_urls"]:
        print(f' {url}')
    else:
        print(" Could not auto-detect LAN IP. Run: ipconfig")

print(f'\n💻 Open on LAPTOP:')

```

```
print(f' {urls['localhost_url']}")
```

```
print(f"\n🔌 USB mode (no Wi-Fi required):")
```

```
print(f' 1) {urls['usb_adb_reverse_command']}")
```

```
print(f' 2) Open on phone: {urls['usb_phone_url']}")
```

```
if protocol == "https":
```

```
print(f"\n⚠️ If you see 'Not Secure' warning:")
```

```
print(f' Click 'Advanced' → 'Proceed' (it's your own certificate)")
```

```
else:
```

```
print(f"\n⚠️ Running on HTTP - mobile sensors may not work!")
```

```
print(f' HTTPS certificates are required for GPS and DeviceMotion APIs.")
```

```
print("\n" + "-" * 60 + "\n")
```

```
# Start server
```

```
if os.path.exists(cert_file) and os.path.exists(key_file):
```

```
print("🔒 Starting HTTPS server...\n")
```

```
app.run(host="0.0.0.0", port=5000, ssl_context=(cert_file, key_file),  
threaded=True)
```

```
else:
```

```
print("⚠️ Starting HTTP server (limited mobile support)...\n")
```

```
app.run(host="0.0.0.0", port=5000, threaded=True)
```